



香港中文大學 (深圳)
The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice Lecture 2: Standard Template Library (STL)

Jingbang Chen
School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen



Course assessment



- We will be covering ~100 coding problems (range from very easy to hard) all over the semester.
 - To get 30% (including CTE 1%) from homework, you should solve **45** problems among them.
 - If you manage to solve it during the class (and present in the class), each problem can be considered as 1.5 problems.
 - Midterm: I will select 20 problems one week before and will test 3 problems (30 points each) from them + 1 new problem (10 points).
 - Final: : I will select 30 problems one week before and will test 4 problems (20 points each) from them + 1 new problem (20 points).
-
- Students can use Python/C/C++/Java to answer the questions in Online Judge (OJ) system.
 - Some problems may have partials, but getting full correct (AC) gives you full score in this problem.



What is STL?

Modern language provides algorithmic APIs to support routine operations, such as sorting, indexing, etc.



Goal

- ▶ Some ability/awareness we should obtain here:
 - Abstract Modeling
 - Separate Algo-Paradigm and Data Structure
 - Complexity Analysis
 - Hyperparameter / API
 - Combine to Achieve More

Goal: We are not learning how to use APIs, but how to reframe/reduce problems.



Sort

- ▶ The key is to customize the comparator
- ▶ E.g. Given (name,score,age), sort score by descend, age by ascend, name by alphabetic

C++

cpp

```
sort(v.begin(), v.end(), [](auto &a, auto &b) {  
    if (a.score != b.score) return a.score > b.score;  
    if (a.age != b.age) return a.age < b.age;  
    return a.name < b.name;  
});
```

Python

python

```
v.sort(key=lambda x: (-x.score, x.age, x.name))
```

Java

java

```
Collections.sort(v, (a, b) -> {  
    if (a.score != b.score) return b.score - a.score;  
    if (a.age != b.age) return a.age - b.age;  
    return a.name.compareTo(b.name);  
});
```



Discuss 1

cpp

```
sort(v.begin(), v.end(), [](auto &a, auto &b) {  
    return a.score >= b.score;  
});
```

► Is this correct?

→ The comparator needs a strictly partial order!

→ Undefined Behavior (UB)



Merge Intervals (15 mins)

- ▶ Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.
- ▶ Two intervals $[a, b]$ and $[c, d]$ are considered **overlapping** if $c \leq b$ (assuming the intervals are sorted by start time), meaning they share at least one common point.



Non-overlapping Intervals (15 mins)

- ▶ Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, return the **minimum number of intervals you need to remove** so that the remaining intervals are **non-overlapping**.
- ▶ Two intervals are considered **overlapping** if they share at least one common point.
That is, intervals $[a, b]$ and $[c, d]$ overlap if $c < b$ (when sorted by start time).



Top-K / Priority Queue

- ▶ Also need to be careful about the comparator (strictly partial order, and the direction)

C++

cpp

```
priority_queue<int, vector<int>, greater<int>> pq;
```

Python

python

```
import heapq  
heap = []
```

Java

java

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```



Discuss 2

- ▶ What's the complexity of these alternative methods:
 - ▶ 1. sort whenever insert
 - ▶ 2. store all with multisets
 - ▶ 3. use `nth_element`
- ▶ → 1. quadratic complexity
- ▶ → 2. large constant
- ▶ → 3. wrong, since it is for a static case.



Merge k Sorted Lists (15 mins)

- ▶ You are given an array of k linked lists, where each linked list is sorted in **ascending order**.
- ▶ Merge all the linked lists into one **sorted** linked list and return it.



Single-Threaded CPU (15 mins)

- ▶ You are given n tasks, where $\text{tasks}[i] = [\text{enqueueTime}_i, \text{processingTime}_i]$.
- ▶ A single-threaded CPU can process **at most one task at a time**, and it follows these rules:
- ▶ If the CPU is **idle** and there are **no available tasks**, it remains idle.
- ▶ If the CPU is **idle** and there are available tasks, it will choose the task with the **smallest processing time**.
- ▶ If multiple available tasks have the same processing time, the CPU chooses the one with the **smallest index** (the original task index).
- ▶ Once the CPU starts processing a task, it runs it **to completion** (no preemption).
- ▶ A task becomes **available** at time enqueueTime_i .
- ▶ Return the **order of task indices** that the CPU will process.



Map (Set) / HashMap (Set)

- ▶ Asymptotic Complexity VS Empirical Efficiency
- ▶ To summarize:
 - 1. Need to be ordered (find prev/succ..), we use TreeMap/map;
 - 2. High Frequency Inquiring, we use HashMap/unordered_map;
 - 3. Static data, we may use sort + binary_search
- ▶ Some insights:
 - HashMap's costs come from the hash conflict, sometimes we need to customize the hash function.



Longest Consecutive Sequence (15 mins)

- ▶ Given an unsorted array of integers `nums`, return the length of the **longest consecutive elements sequence**.
- ▶ A consecutive sequence means numbers that increase by exactly 1 each time (e.g., 3, 4, 5, 6). The sequence does **not** need to be contiguous in the original array.



Binary Search

- ▶ After sorted, we can use these APIs to find the first/last elements of some conditions. (Don't use it wrong!!!!!!)

语义	C++	Python	Java
$\geq x$	<code>lower_bound</code>	<code>bisect_left</code>	<code>ceilingKey</code>
$> x$	<code>upper_bound</code>	<code>bisect_right</code>	<code>higherKey</code>



Discuss 3

- ▶ How to remove all duplicate elements and assign each element its new rank?
- ▶ How to remove all duplicate elements but keep the initial order (of the first appearance)?



Intelligence Test (20mins)

You are given a sequence of integers:

$$A = (a_1, a_2, \dots, a_n)$$

Then you will be asked q queries.

In each query, you are given another sequence:

$$B = (b_1, b_2, \dots, b_m)$$

For each query, determine whether B is a **subsequence** of A , i.e., whether it is possible to delete some elements from A (possibly none) so that the remaining elements form exactly B in order.

Return **YES** if B is a subsequence of A , otherwise return **NO**.



One more problem...



Sliding Window Maximum (20mins)

- ▶ You are given an array of integers `nums`, and an integer `k`.
- ▶ There is a sliding window of size `k` which moves from the left to the right of the array. At each step, the window shifts right by one position.
- ▶ Return an array `ans`, where `ans[i]` is the **maximum value** in the `i`-th window.
- ▶ **Bonus: What if we need to maintain the median?**